

What Archie is, and what it does on the computer it runs on

A plain description of the software for anyone who has been asked to approve it: where it installs, what it sends over the network, where credentials are kept, and what it is not allowed to do.

PRODUCT

Archie, version 0.1.7 (beta)

PUBLISHER

Otian AI LLC

PLATFORMS

macOS and Windows

DOCUMENT DATE

August 25, 2026

CONTACT

**jett@otianai.com
jack@otianai.com**

WEBSITE

otianai.com

SECTION 1

What Archie is

Archie is a desktop application that lets one person set up an AI assistant and then talk to it from a chat app they already use.

It is installed like any other desktop program and it runs only while it is open. There is no Archie server that holds the assistant: the assistant is a process on the computer where the app is installed. Close the app and it stops answering.

The person using it does three things. First they connect an **AI account**. This is an API key from a provider such as Anthropic, OpenAI, or Google: a developer account billed by what it is used for, not a consumer chat subscription like ChatGPT Plus or Claude Pro. The difference matters for a review and section 13 sets it out, but the short form is that the major providers do not train on anything sent through their API, and that is the default rather than a setting somebody has to find. Second, they optionally connect a chat app, such as Telegram, Slack, Discord, Signal, or Matrix, so they can reach the assistant from their phone in addition to their computer. Third, they install **skills**, which are small bundles of instructions that tell the assistant how to handle one kind of job, like drafting a reply to an email or putting an event on a calendar.

What the assistant can actually touch is decided entirely by what the user connects. A fresh install can do nothing except talk. Reading a mailbox requires that person to sign in to that mailbox and grant permission. Reading a calendar is a separate permission. Nothing is on by default and nothing is inherited from another user.

THE SHORT VERSION FOR A REVIEWER

Archie is a locally installed client. It makes outbound HTTPS connections to the services the user connected and to the vendor's own catalog and license check. It opens no listening network port, accepts no inbound connection, runs no shell command, and installs no service, driver, or kernel extension. It does not require administrator rights to install on Windows.

SECTION 2

How the application is built

Archie is three parts, each allowed to do less than the one below it. The split exists so that the part handling untrusted text is the part furthest from the credentials.

The window

NO SECRETS

The screens people look at, written in React and TypeScript and drawn by the operating system's own web view. It holds no credentials, runs no database queries, and cannot read or write files. It asks the layer below for everything, through a fixed list of named commands that is deny-by-default.

↓ **typed commands**, allowlisted one by one

The core

HOLDS THE KEYS

Rust. Owns the local database, the operating system credential store, and the audit log. This is the only part that can read a secret value, and it reads one only at the moment it is putting it into a specific outgoing request. A secret is never handed back up to the window after it is typed in.

↓ **standard input and output only**, no network port

The runtime

RUNS THE ASSISTANT

A separate child process that does the actual work: deciding which skill a message belongs to, calling the AI provider, and running tools. It is a separate process so that a hang or a crash can be restarted without taking the app down with it. It talks to the core over the pipe it was started with, not over the network.

The application is written in Rust and TypeScript and packaged with Tauri 2, which means it uses the operating system's built-in web view rather than shipping a copy of a browser.

SECTION 3

What gets installed, and where

PLATFORM	INSTALLER	RIGHTS NEEDED
macOS	A signed disk image (<code>.dmg</code>). The user drags the app into Applications.	None beyond the ability to write to Applications. It can also be run from the user's own folder.
Windows	An NSIS installer (<code>.exe</code>), built in current user mode.	None. It installs under the signed-in user's own profile and does not write to Program Files or to the machine-wide registry hive.

No background service, scheduled task, daemon, driver, or kernel extension is installed. Nothing runs at startup unless the user turns that on. When the app is closed, no Archie process remains.

Files it creates

WHAT	WHERE ON MACOS	HOW IT IS STORED
Settings, agent configuration, conversation history	<code>~/Library/Application Support/com.archie.app/archie.db</code>	A SQLite file. Not separately encrypted. See the note below.
Credentials and API keys	macOS Keychain, service <code>com.archie.app</code>	Operating system credential store. Never written to the database or to any file.
Logs	<code>~/Library/Logs/Archie/</code>	Rotating JSON lines. Every line passes a redactor that strips secret-shaped values.
Caches and files the assistant saved	<code>~/Library/Caches/com.archie.app/</code>	Ordinary files.

On Windows the same four live under the user's `AppData` folder, and credentials live in Windows Credential Manager instead of the Keychain.

WORTH KNOWING

The local database is a plain SQLite file. It holds settings, the assistant's configuration, and conversation history, and anyone who can read that user's profile folder can read it. It never holds an API key, a password, or an OAuth token. If conversation history on the machine is a concern, the control is whole-disk encryption: FileVault on macOS or BitLocker on Windows.

SECTION 4

Network behavior

Every connection Archie makes is outbound HTTPS or a secure websocket that it opened itself. Nothing on the network can connect to it.

Inbound: none

Archie listens on no port and accepts no inbound connection. There is one narrow exception, and it lasts seconds:

- When the user signs in to Google, Microsoft, or their Otian account, the app opens a listener on `127.0.0.1` on a port the operating system picks, because that is how the standard OAuth desktop flow returns the user to the app after they approve in their browser. It is bound to the loopback address, so it is not reachable from the network, and it closes as soon as the browser hands back the response.

The runtime that executes the assistant has no socket of its own at all. It communicates with the rest of the app over standard input and output.

Local network: only if the user asks for it

Archie has an optional feature for switching smart lights and plugs on the user's own wifi. It is off until the user runs a scan and adds a device by hand. If it is used, the app sends UDP discovery broadcasts on the local subnet. On a corporate network this feature has nothing to find and can simply be left alone; there is no need to use it.

SECTION 5

Where it connects, and why

This is the full list of destinations, grouped by what causes the connection. Anything in the second group happens only because the user connected that service.

Always, as part of the application working

HOST	PURPOSE	NOTES
otianai.com	Sign-in, license check, update manifest	The vendor's own site. Also serves the optional phone page.
*.googleapis.com	Firebase Auth and Firestore	The vendor's Firebase project holds the account record, the add-on catalog, and the license tier. This is Google infrastructure, not Google user data.
github.com	Downloading an app update	Update binaries are GitHub release assets. Every update is signature-verified before it is applied.
api.stripe.com, checkout.stripe.com	Subscription and billing	Opened in the browser. Archie never sees a card number.

Only after the user connects that service

HOST	WHAT CONNECTS TO IT
api.anthropic.com, api.openai.com, generativelanguage.googleapis.com, api.groq.com, and similar	The AI provider whose key the user pasted in. This is where the assistant's thinking happens.
gmail.googleapis.com, www.googleapis.com, tasks.googleapis.com, oauth2.googleapis.com	Gmail, Google Calendar, and Google Tasks, if the user signs in to a Google account.
graph.microsoft.com, login.microsoftonline.com	Outlook mail and calendar, if the user signs in to a Microsoft 365 account.
api.telegram.org, discord.com, slack.com, the user's Matrix homeserver, Signal's servers	Whichever chat app the user connected, so the assistant can receive and send messages there.
A per-service host, one host per key	Any other API the user connects, such as Notion or Todoist. A key pasted for one service is bound to that service's host and cannot be sent anywhere else.
Arbitrary public web pages	Only if the user installs a skill that reads web pages or runs a research task. Private, loopback, and carrier-grade NAT addresses are refused. See section 9.
The vendor's asset host	Large optional downloads, such as the speech model or a media tool, if the user installs an add-on that needs one. Each one is checked against a published SHA-256 before it is used.

IF YOU WANT A FIREWALL ALLOWLIST

The minimum for the app to start and stay licensed is `otianai.com`, `*.googleapis.com`, and `github.com`. Everything else can be added or withheld per service, and withholding one disables that feature rather than breaking the app.

SECTION 6

What stays on the computer, and what leaves it

Never leaves the computer

- API keys, OAuth tokens, and every other credential. They stay in the operating system credential store and are read only to be put into one specific outgoing request.
- The conversation history and the assistant's memory.
- The assistant's configuration, skills, and knowledge base files.
- Voice. Speech is turned into text on this computer and spoken replies are synthesized on this computer. No audio is uploaded.
- Anything about the local network, including any smart device the user added.
- Logs and the audit trail.

Leaves, and to whom

- **To the user's own AI provider:** the text of the current conversation, plus whatever the assistant needed to answer, such as the body of an email it is drafting a reply to. This goes on the user's own account and key, direct from this computer.
- **To the connected service:** ordinary API calls, such as reading a message from Gmail or creating a calendar event.
- **To the chat app:** the messages, exactly like any other message that app carries.
- **To the vendor:** the account email, the plan check, and the version heartbeat described in section 10. The complete list of what Otian's servers hold is below, and on the free trial credits the AI calls themselves pass through Otian's server (section 7).

THE POINT WORTH PULLING OUT

Once an AI account is connected, message content never passes through a server belonging to Otian AI. There is no relay, no proxy, and no queue in the middle: the computer running Archie talks to the AI provider directly, on the user's own key. That is also why the assistant stops working when the app is closed.

The one exception is the free trial credits a new install starts on, which do route AI calls through Otian's server. Section 7 covers what that means and what the trial refuses to install because of it. Connecting an AI account ends it.

Everything Otian AI's servers hold

Not a summary. This is the whole list, and it is the honest maximum a legal demand to Otian AI could produce. None of it is the content of anybody's work.

WHAT

Your email address and account id

WHEN IT EXISTS

Always. It is how an account is identified.

WHAT	WHEN IT EXISTS
Whether you have a current plan	Always.
Which paid add-ons you bought	Per purchase, kept permanently as a receipt so a paid add-on can be reinstalled on another computer. No per-person record is kept of the free add-ons anybody installs.
A version heartbeat	App version, platform, edition, last seen. One record per account. Can be switched off.
Crash tails	Only if crash reporting is left on, which it can be switched off.
The trial-credit ledger and its spend history	Only for an account that took the free credits: token counts and amounts, with salted hashes of the device and the IP address so the grant limits can be enforced. Never message content.
Second-factor records	Only if two-factor sign-in is set up.
Invoices and refused-checkout records	Only where a payment or a refused payment happened. A refusal record is the account id, the country, and the amount.
Sealed phone messages, plus their timestamps and delivery status	Only if phone access is turned on. The messages are ciphertext under a key this computer generated and never sent, so Otian holds them and cannot open them. The timestamps are not encrypted.

What is not on that list is the part that matters: no conversation, no email, no calendar entry, no file, no memory, and no record of what any assistant was asked or did.

Checking on it from a phone, which is opt in

There is a feature for checking on an assistant from a phone at otianai.com/phone. Because the assistant lives on the one computer, the phone leaves a command in a mailbox on the vendor's Firestore and the computer picks it up. Both ends encrypt their side with AES-256-GCM under a key that this computer generates and hands to the phone by QR code. What the vendor's database holds is ciphertext that nobody at Otian AI can open. This is off unless the user turns it on, per computer.

SECTION 7

The free trial, and what it refuses to do

A new install can talk to an assistant before anybody pastes a key, because Archie grants two dollars of Anthropic usage, paid on Otian's own account, for up to fourteen days. That trial is the one configuration in which an AI call passes through a server belonging to Otian AI, and it is fenced accordingly.

The credits exist so somebody can see Archie work before being asked for an account or a card. During the trial, calls travel from this computer to Otian's billing service and on to the model, because the alternative was shipping Otian's own AI key inside the app, where anyone could pull it out. Two dollars is enough to try the product and not enough to run on: the credits are a trial of Archie, not a license to it.

On the trial credits, the agent cannot read email at all

This is not a policy about which add-ons may be installed. It is a gate in the running code. **Every tool whose name begins with `inbox_` or `watch_` is refused at the moment the tool is dispatched**, before any mail code runs, and the background mailbox watcher refuses on the same condition. The arm sits ahead of every mail branch on purpose, so a mail tool added to Archie next year is born gated rather than born as a hole. The agent says why in plain words instead of failing quietly.

A second layer sits at install time, over the add-ons that hold a legally distinct kind of information: health records, medication reminders, the text-message add-on, and the two school add-ons, which read a parent's own school mail and so end up holding details about a minor. Each carries a field naming the reason it needs an AI account of the owner's own, and that sentence is what the person is shown when the install is declined.

That last one is worth a sentence of its own, because the word invites a question this document should answer rather than leave hanging. **Archie is one adult's assistant**. There is no edition for a household and none for anybody under eighteen, no child signs in to it or talks to it, and the school add-ons are a parent reading mail addressed to them. The category is protected here for the same reason health information is: it is about somebody who never chose this product.

Neither layer is left to a policy somebody remembers. Two tests in the codebase enforce the second one: the build fails if an add-on that looks like it holds health information or a minor's does not carry the field, and it fails again if the reason is too short to act on or does not tell the reader what to do instead.

WHY THIS IS A RULE AND NOT A SETTING

Nothing is logged on that server, and Otian could not read a mailbox through it even if somebody wanted to. That is not the point. Mail is written by people who never chose this product and were never asked, and being the machine their words pass through is not a thing to be, in transit or otherwise. A company that would route a stranger's email through its own server because it technically could is a company that has already decided the answer to every question after that one.

Connecting an AI account removes the middle entirely. Every call then goes straight from this computer to the provider on the user's own key, nothing about the work reaches Otian AI, and every email tool works with nothing to switch on. That is the configuration the rest of this document describes, and it is the one anybody using Archie for work should be on.

There is a second trial for exactly that reason: fourteen days on an AI account of your own, offered when the credits have already been claimed on a computer. Nothing passes through Otian on it. For an organization evaluating Archie, that is the trial to ask for, because it is the same configuration you would be approving for permanent use.

SECTION 8

Accounts and the permissions it asks for

Mail and calendar access uses the standard OAuth flow in the user's browser. Archie never sees the password, and the permission screen is the provider's own.

The user ticks what they want on a chooser before the browser opens, so a person who wants calendar only never grants mailbox access. These are the scopes that can be requested:

PROVIDER	SCOPE	WHAT IT ALLOWS
Google	<code>gmail.readonly</code> or <code>gmail.modify</code>	Read mail. <code>modify</code> is requested only when the user wants the assistant to file mail out of the inbox, which is reversible.
	<code>gmail.compose</code>	Create a draft and send it. See the note below.
	<code>calendar.readonly</code> , <code>calendar.events</code>	Read the calendar, and create or change events.
	<code>tasks</code>	Google Tasks, if the user uses to-do features.
	<code>userinfo.email</code> , <code>drive.file</code>	Identify which account was connected. <code>drive.file</code> covers only files the user picked in the Google file picker, never the whole Drive.
Microsoft	<code>Mail.Read</code> or <code>Mail.ReadWrite</code>	Read mail; <code>ReadWrite</code> for the same filing feature.
	<code>Mail.Send</code>	Send as the user. See the note below.
	<code>Calendars.Read</code> , <code>Calendars.ReadWrite</code>	Read the calendar, and create or change events.
	<code>User.Read</code> , <code>offline_access</code>	Identify the account, and refresh the token without asking the user to sign in again every hour.

READ THIS ONE CAREFULLY, BECAUSE IT IS THE STRONGEST PERMISSION ON THE LIST

Send access is real access. What bounds it in Archie is the design of the mail features rather than the scope: the background mailbox watcher is **allowed to read and propose only**, and a drafted reply is sent when, and only when, a person presses Send on it. There is no setting that lets it send on its own, and no tool the AI can call that reaches the send path unattended.

That is a property of the software, not of the token. The token, once granted, permits sending in the way any mail client's token does. A reviewer who is not comfortable with that should have the user leave the send permission unticked on the consent screen. Mail features other than sending still work.

A further protection is worth naming, because it addresses the risk people have in mind when an AI is pointed at a mailbox. **When a model is reading an untrusted email, it is given no tools at all.** An email that tries to instruct the assistant, for example "search my mail for the password reset and quote it back", has nothing to instruct: there is no tool in the model's hands at that moment. The optional lookups a drafter may perform are named from a fixed three-item list, and every parameter of the resulting query is built by the app from facts it already holds, such as the sender's address and today's date. The model can ask for a door that exists; it cannot describe a new one.

What the AI is not allowed to do

These are structural limits in the code, not instructions in a prompt. A prompt can be talked around. These cannot.

LIMIT	HOW IT HOLDS
No shell, no code execution	There is no tool that runs a command, evaluates code, or loads a plugin. Tools are a fixed list declared in the source with typed inputs that are validated before the call runs. Model output is never turned into a command line. Installing anything through <code>pip</code> , <code>npm</code> , or <code>brew</code> is refused outright, because those cannot be pinned or verified.
No reaching inside the network	The web tools refuse any private, loopback, or carrier-grade NAT address, and refuse any hostname that resolves to one. The check runs inside the HTTP client's own name resolver, so a DNS server that answers differently the second time has no second lookup to win. This is a standing defense against server-side request forgery, and it means the assistant cannot be talked into fetching an internal page.
No capability it was not given	The tool list is assembled fresh on every turn from that one assistant's own configuration. There is no inherited or ambient capability, and one assistant cannot read another's memory or credentials. Every key is bound by name to the assistant that holds it, and connector keys are additionally bound to a single host.
Nothing that changes the outside world without a person	Calendar changes, to-do changes, sending mail, and writes to a connected service all stage as a pending change and are shown to the user for confirmation. The confirmation is the commit.
No silent history	Security-relevant actions are written to an append-only log where each entry's hash chains from the one before it, so a deleted or edited entry is detectable. The chain's tip is anchored in the operating system credential store and verified at startup, which catches truncation of the end of the log. Everything written passes the redactor first.
No unverified binary	Some optional add-ons download a helper program, such as ffmpeg or a speech model. An add-on can only reference a program by an opaque identifier, never by a URL, so it can point at a vetted binary and can never introduce one. The registry mapping identifier to file is the vendor's own, built from pinned sources. Every download is checked against its published SHA-256 before it is made runnable, so a hijacked bucket or an interrupted transfer leaves nothing executable behind.

Two features go further than the rest because they touch other people's words, and both are worth stating plainly. The mailbox watcher may read and propose only. The optional macOS text-message feature reads only forward from where it last stopped, never scans history on its own, requires Full Disk Access, which no application can grant itself, and can reach the send path through exactly one private function driven by one button a person pressed. A test in the codebase counts the call sites of that function and fails the build if a second one ever appears.

SECTION 10

Telemetry, and how to switch it off

Archie collects two things. This is the whole list.

WHAT	CONTENTS	HOW OFTEN
Heartbeat	App version, platform, edition, and the time it was last seen. One record per account.	At most once every six hours.
Error report	The tail of a crash, and whether the app is stuck restarting. Queued to disk so a crash can report itself on the next launch rather than during the crash.	Only when the app crashes.

There is no feature usage tracking, no analytics of any kind, no counts of messages or actions, and no message content. Every string in a report passes the same redactor that guards the logs. The purpose is narrow and worth stating: a desktop app is the one place a bad release cannot be pulled back, because the broken copy is on someone else's computer, so the vendor needs to know which versions are actually running and whether one of them is crashing.

It can be turned off, in Settings, under Crash reports. The same screen lists what a report contains. Turning it off stops the heartbeat, stops the uploads, and deletes anything already queued.

The remote switches, and what they cannot do

The vendor can, without shipping a new release, switch off a broken add-on, switch off a messaging channel, post a notice in the app, or set a minimum version below which assistants refuse to start. This is a containment mechanism for a bad release. It cannot read anything, cannot reach a file, and cannot make the app do something it does not already do. Every default is permissive: a missing document, an absent field, and an unreachable server all behave identically, which means a network that blocks the vendor entirely does not hand anyone a lever.

SECTION 11

Permissions the operating system will ask about

Each of these produces a system prompt the user must accept. None is required for the app to run, and each corresponds to one feature.

PROMPT	PLATFORM	WHAT IT IS FOR, AND WHAT HAPPENS IF IT IS DENIED
Microphone	macOS and Windows	Talking to the assistant instead of typing. Speech is turned into text on this computer and never uploaded for transcription. Denied: the microphone button does not work, and nothing else changes.

PROMPT	PLATFORM	WHAT IT IS FOR, AND WHAT HAPPENS IF IT IS DENIED
Local network	macOS 15 and later	Finding smart lights and plugs on the user's own wifi. Denied: the device scan finds nothing, and nothing else changes. On a corporate network, denying it is a reasonable default.
Full Disk Access	macOS only	Required only for the optional feature that reads the Mac's Messages database to propose replies to texts. The user must grant it by hand in System Settings; no application can grant it to itself. Denied, or simply not granted: the text feature does not run, and nothing else changes.

Archie deliberately does not request Contacts access. Note also that the macOS build carries two hardened-runtime relaxations, `disable-library-validation` and `allow-dyld-environment-variables`. They exist because one optional bundled media tool is packaged with PyInstaller and unpacks its own Python libraries at launch, which library validation would otherwise reject. This is the standard entitlement set for shipping that kind of notarized binary, and its effect is scoped to what the app itself can load.

SECTION 12

Signing, distribution, and updates

PLATFORM	SIGNED	DETAILS
macOS	Yes, and notarized	Signed with an Apple Developer ID certificate issued to Jett Nguyen, team ID <code>5SLXY6F2CD</code> , then submitted to Apple for notarization and stapled. Gatekeeper opens it without a warning. The signature can be checked with <code>codesign -dv --verbose=4</code> and <code>spctl -a -vv</code> against the downloaded app.
Windows	No	The installer is not code-signed, so SmartScreen shows a warning on first launch. See the note below.

STATED PLAINLY, BECAUSE YOU WOULD FIND IT ANYWAY

The Windows installer ships unsigned. This is a gap and it is known. It is not a cost decision: the affordable route to a code-signing certificate requires three years of company history, and Otian AI LLC is not old enough yet. The practical effect is that a Windows user meets a SmartScreen warning and has to click through it, which is exactly the kind of thing an IT department is right to treat as a red flag when it appears unexplained.

What can be verified instead: the download comes from a known GitHub release under the publisher's own account, the file hash can be compared with the published release, and the macOS build of the same application is fully signed and notarized by Apple. If an unsigned installer is not acceptable under policy, the macOS build is the one to approve.

Application updates are checked at launch against a manifest, downloaded from GitHub, and **verified against a signing key compiled into the app before they are applied**. An update that fails that check is not installed. The update is offered to the user rather than applied silently, and requires no administrator rights.

What the AI provider does with the data

Archie does not choose this. The user pastes an API key for a provider they have an account with, and that provider's terms govern what happens to the text.

Because that difference lands on whoever pasted the key, each provider's own wording is kept on file, quoted rather than paraphrased, with the page it came from and the date it was read. Twice-yearly re-reading is part of the arrangement, because a stale quote on a page whose whole worth is that it can be checked is worse than no quote. This is the current summary, read on August 16, 2026:

PROVIDER	TRAINS ON WHAT YOU SEND?	THEIR OWN PAGE
Claude (Anthropic)	No	anthropic.com/legal/commercial-terms
ChatGPT (OpenAI)	No, unless you opt in. Abuse-monitoring logs are kept up to 30 days.	developers.openai.com/api/docs/guides/your-data
Groq	No	console.groq.com/docs/legal/services-agreement
Gemini (Google)	Free key: yes. Paid key: no.	ai.google.dev/gemini-api/terms
Mistral	Free mode: yes by default. Pay as you go: no.	legal.mistral.ai/terms/commercial-terms-of-service
DeepSeek	Yes, and the data is stored in China, if you use DeepSeek's own hosted service. Their models are open weight, so a copy running on hardware you control sends them nothing.	cdn.deepseek.com/policies/en-US/deepseek-privacy-policy.html
Grok (xAI)	Unknown. Their legal pages return an error to every automated reader, so there is no quote to stand behind.	x.ai/legal (unreadable as of the date above)

How an API key compares with an enterprise plan

This is the question most reviewers actually want answered, and the honest answer has two halves.

On what happens to the text, an API key gives you what an enterprise plan gives you. Anthropic, OpenAI, and Groq all state that content sent through their API is not used to train models, and that is the default for an ordinary paid key rather than a term somebody has to negotiate. It is the consumer chat subscription that is the weaker product here: those may train on conversations, which is exactly why Archie is built around an API key and not around a chat login.

On what you can prove and enforce, it does not. An enterprise or business agreement is a contract, and it carries things a self-service API key does not:

WHAT ENTERPRISE ADDS	WHY AN API KEY ALONE DOES NOT HAVE IT
A signed data processing agreement	Standard API terms are accepted by clicking, not signed. If your policy requires a countersigned DPA naming your organization, that comes with the commercial agreement.
Zero data retention	OpenAI keeps abuse-monitoring logs for up to 30 days on ordinary API use. Removing that is a separate arrangement you apply for, not a default.
Compliance artifacts	SOC 2 reports, a HIPAA business associate agreement, and data residency commitments are provided under a commercial agreement, not attached to a key you generated on a web form.
Administrative control	Single sign-on, seat management, spending limits, and the ability to revoke one person's access centrally belong to the organization account. A personal key belongs to the person who made it.

THE PRACTICAL ANSWER FOR AN ORGANIZATION

If your organization already has an enterprise or team agreement with Anthropic, OpenAI, or another provider, issue an API key from that account and have the user paste that key. Archie treats it like any other key, the traffic goes straight from the computer to your provider, and every guarantee in your existing contract applies to it without Archie being party to any of it. That is the strongest configuration available and it needs nothing special on Archie's side.

Running the model on your own hardware

Archie also offers a provider option called **Another provider**, which takes any endpoint speaking the widely used OpenAI-compatible format along with the model name to ask for. That covers a model your organization hosts itself. Under that arrangement the text never reaches any AI vendor: it goes from this computer to a machine you operate. For a team that cannot send its work to a third party at all, this is the route, and it is worth naming even though most people will not take it.

SECTION 14

Open points, stated rather than buried

Archie is at version 0.1.7 and is in beta. A document that lists only the good parts is not useful to someone deciding whether to trust it, so here is the rest.

POINT	WHAT IT MEANS FOR A REVIEWER
The Windows installer is unsigned	Covered in section 12. Users meet a SmartScreen warning. The macOS build is signed and notarized.
It is beta software	The version number is honest. Every feature described here works and has been exercised, but the product has not been through a formal external penetration test.

POINT

WHAT IT MEANS FOR A REVIEWER

The update feed lives at an unlisted address

During the beta the update manifest is at an unguessable URL rather than behind an access check. The URL is readable out of any copy of the app, so this is obscurity and not access control. What actually protects the update path is the signature check on the downloaded package, which is real. Replacing the unlisted URL with a proper gate is on the list.

The local database is not separately encrypted

Conversation history sits in a SQLite file readable by anyone with access to that user's profile. No credential is in it. Whole-disk encryption is the control.

Mail scopes are real scopes

The propose-only behavior is enforced by the application. The token itself permits what the user granted. Leaving the send permission unticked is the way to remove that entirely.

Some add-ons run helper binaries

Optional media and speech tools run as child processes with an explicit argument list, a deadline, and a scratch directory that is deleted afterward. They are hash-verified before they can run. They are not sandboxed by the operating system, and the document above says so rather than implying otherwise.

REMOVING IT

Archie is a normal application and uninstalls like one. Deleting the app removes the program; deleting the folder named in section 3 removes its data; the credentials it created appear under `com.archie.app` in Keychain Access or Windows Credential Manager and can be deleted there. Access already granted to a mailbox or calendar should be revoked at the provider, in the Google account permissions page or the Microsoft 365 admin center, which is where OAuth grants actually live.

Questions. Anything in this document can be checked further, including specific network captures, the signing details on a given build, or a walkthrough of a feature before it is approved. Write to Jett at jett@otianai.com or Jack at jack@otianai.com.

Archie 0.1.7 · Otian AI LLC · otianai.com · Prepared August 25, 2026. Details describe this version and may change in later releases.